

Czy Ballista naprawdę liczy rozproszenie?

Etap P0 wytłumaczony od podstaw — co sprawdziliśmy, co wyszło i co to znaczy dla pracy

Miłosz Kowalewski

30 września 2026

O co chodziło

Z poprzedniego etapu wiedzieliśmy, że pięć operacji genomicznych daje w Ballistcie **poprawne wyniki**. Nie wiedzieliśmy jednak na pewno, czy są liczone **rozproszenie** — czyli przez kilka niezależnych programów, które wymieniają się danymi.

Wracając do analogii z kuchnią: do tej pory szef kuchni i kucharz byli **jedną osobą**. Przepis był zapisywany na kartce i odczytywany, dane były tasowane — ale wszystko działało się w jednej głowie. Mocna przesłanka, ale nie dowód. Etap P0 miał to sprawdzić porządnie: **szef i dwóch kucharzy jako trzy osobne osoby, w osobnych pokojach, porozumiewające się wyłącznie przez podawanie kartek**.

To ważne, bo cała część naukowa pracy — pomiary „czy rozproszenie się opłaca” — ma sens tylko wtedy, gdy rozproszenie naprawdę zachodzi.

Co zbudowaliśmy

- **Program ballista_node**, który uruchamia albo „szefa kuchni” (scheduler), albo „kucharza” (executor) jako osobny program. Złożyliśmy go z oficjalnych części Ballisty — samej Ballisty nie zmienialiśmy, zgodnie z założeniem pracy.
- **Tryb zdalny klienta** — nasz program z zapytaniem łączy się teraz z szefem kuchni działającym gdzie indziej, zamiast uruchamiać całą kuchnię u siebie.
- **Zestaw testów**, który sam stawia taką kuchnię z trzech programów, zleca pięć operacji i sprawdza dowody.

Po drodze okazało się, że osobny szef kuchni musi sam przygotować sobie „warsztat” (bioową konfigurację), który wcześniej dostawał po cichu od klienta, i że trzeba mu wprost pozwolić przyjmować ustawienia Ballisty od klienta — inaczej po cichu je ignorował.

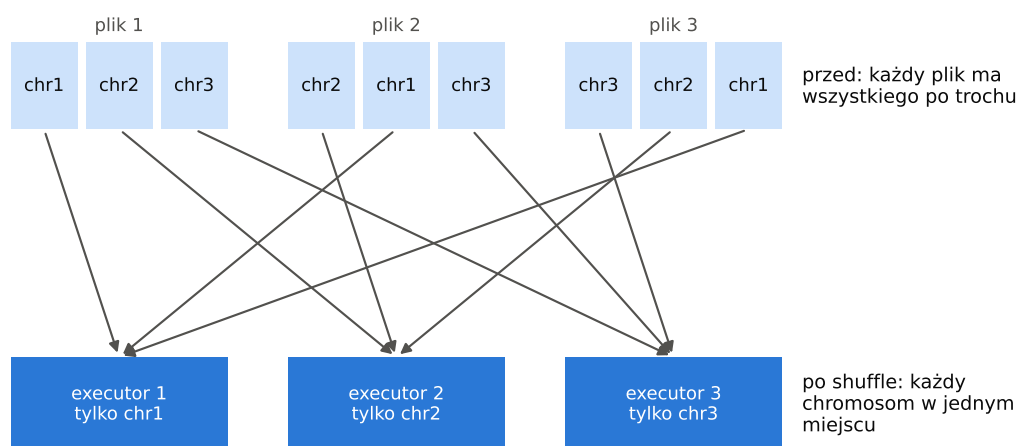
Cztery dowody — prostym językiem

1. **To naprawdę osobne programy.** Każdy ma inny numer w systemie, a szef kuchni widzi dwóch różnych kucharzy.
2. **Wyniki są poprawne** — takie same jak z polars-bio na jednym komputerze.
3. **Obaj kucharze pracowali.** Każdy kucharz odkłada wyniki swoich kroków na własną półkę (osobny katalog na dysku). Po zapytaniu zaglądamy na obie półki i widać wprost, kto co zrobił. Jeśli na obu półkach leżą wyniki tego samego kroku tasowania, dane musiały przejść od jednego kucharza do drugiego.
4. **Kontrola negatywna.** Jednemu kucharzowi zabieramy „tłumacza” (kodery), czyli umiejętność czytania naszych niestandardowych przepisów. Jeśli kucharz naprawdę sam czyta przepis, musi się na nim wyłożyć.

Co wyszło

Operacja	Obaj kucharze pracowali?	Dane przeszły między nimi?
merge	tak	tak
subtract	tak	tak
nearest	tak	nie dotyczy (każdy dostaje kopię małej tabeli)
coverage	tak	nie dotyczy (jw.)
overlap	nie — jeden	nie dotyczy

Przypomnienie, jak działa tasowanie (shuffle) — to ono jest „przejściem danych między kucharzami” w merge i subtract:



Rysunek 1: Shuffle: na starcie każdy plik ma wszystkiego po trochu; po przetasowaniu każdy chromosom trafia w jedno miejsce.

overlap liczy jeden kucharz, bo przy obecnych danych ta operacja ma tylko jedno zadanie na krok — to znane ograniczenie z wcześniejszego etapu i dobry kandydat do dalszej pracy.

Kontrola negatywna zadziałała: kucharz bez tłumacza zrobił pierwszy, zwykły krok, ale na kroku z naszą operacją zgłosił, że nie potrafi odczytać przepisu. To jest dowód, że przepis czyta naprawdę on, a nie ktoś za niego.

Niespodzianki

- **Ballista w takiej sytuacji nie zgłasza błędu, tylko czeka.** Gdy kucharz nie umie odczytać przepisu, szef kuchni uznaje go za „zaginionego”, a gdy kucharz zgłosi się ponownie — próbuje jeszcze raz, w kółko. Zapytanie wisi zamiast się zakończyć. To ciekawa informacja do porównania silników pod kątem odporności na błędy.
- **Kolejność startu ma znaczenie.** Kucharz próbuje zgłosić się do szefa tylko raz — jeśli szef jeszcze nie przyszedł do pracy, kucharz rezygnuje.
- **Szef kuchni po cichu ignorował ustawienia Ballisty** przysyłane przez klienta, dopóki nie dostał wprost „słownika” tych ustawień. Gdybyśmy tego nie wyłapali, późniejsze strojenie Ballisty w pomiarach nie miałyby żadnego efektu — a my byśmy o tym nie wiedzieli.

Co to znaczy dla pracy

- **Główne założenie pracy wytrzymało próbę:** da się rozproszyć obliczenia polars-bio w Ballistcie bez przerabiania samego silnika — także wtedy, gdy programy są naprawdę oddzielne.
- **Pomiary będą wiarygodne:** zostaną wykonane na kuchni z osobnych programów, a nie na jednej osobie grającej wszystkie role.
- **Sposób udowodnienia rozproszenia** (cztery dowody, w tym kontrola negatywna) to materiał na osobny fragment pracy — zwykle pokazuje się tylko, że wynik jest poprawny, a to rozproszenia nie dowodzi.
- **Niespodzianki są wynikiem samym w sobie** — zasilają porównanie Ballisty z Sailem w kategoriach „co trzeba umieć, żeby to uruchomić” i „jak silnik zachowuje się przy błędach”.

Czego ten etap jeszcze nie pokazuje

- Wszystko działało na jednym laptopie — bez prawdziwej sieci między komputerami.
- Dane były małe (po pięć przedziałów) — nic tu nie mówi o szybkości.
- Sail wciąż działa lokalnie jako jeden program — to czeka na osobną próbę z Kubernetesem.

Co dalej

1. Pobranie prawdziwych danych z publikacji polars-bio i przejście na szybszy format plików (Parquet).
2. Narzędzie pomiarowe — dopiero ono zacznie mierzyć czasy i pamięć.
3. Próba uruchomienia Saila jako naprawdę rozproszonego.